TD Algorithmique : FPTAS

On considère le problème 2| |L_{max} qui consiste à ordonnancer un ensemble de N tâches indépendantes (i=1,2,3…N) sur 2 machines parallèles identiques. Chaque tâche i possède une durée opératoire p_i et une durée de latence q_i. L'objectif est de trouver un ordonnancement permettant de minimiser la date de livraison maximale $L_{max} = \max_{i=1,2,\ldots,N} \{C_i + q_i\}$ avec C_i la date de fin d'exécution de la tâche i. La préemption est interdite.

1) En observant que la solution optimale est une partition des tâches (en deux sous-ensembles séquencés dans l'ordre de Jackson, c'est-à-dire, l'ordre décroissant des q_i) où chaque partition doit être ordonnancée sur une machine, proposer un algorithme de programmation dynamique *A* pour résoudre ce problème.
   Indications : Essayer de vous inspirer des exemples vus en cours. On pourra prendre comme états générés par l'algorithme des vecteurs $[t, f]$ avec t la date de fin de la dernière tâche sur la première machine et f la date de livraison maximale de toutes tâches ordonnancées.

2) Proposer des limites supérieures pour t et f pour toute solution réalisable.
   Expliquer comment on peut éventuellement améliorer la complexité de la programmation dynamique.

3) On considère l'heuristique H consistant à ordonnancer toutes les tâches sur la première machine. Montrer que $H \leq 3.OPT$ (avec OPT la valeur de la solution optimale).

4) En convertissant la programmation dynamique, montrer que le problème admet un FPTAS (Fully Polynomial Time Approximation Scheme).
   Indications : Il faudrait considérer des états approchés $[t^{\#}, f^{\#}]$ générés par une version tronquée de l'algorithme de programmation dynamique, appelée $A^{\#}$. On pourra montrer qu'à chaque itération i de ces algorithmes, un état $[t, f]$ généré par A possède un état homologue $[t^{\#}, f^{\#}]$ généré par $A^{\#}$ avec les conditions suivantes :

$$t - i.\delta_1 \leq t^{\#} \leq t$$

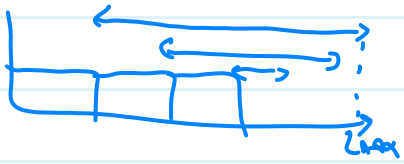
$$f^{\#} \leq f + i.\delta_2 + i.\delta_1$$

   avec $\delta_1 = \varepsilon P/4N$ et $\delta_2 = \varepsilon H/6N$ (noter que P est la somme de toutes les durées opératoires et que ε est la précision de l'algorithme $A^{\#}$).

   Dans la phase de remplacement des états $[t, f]$ par $[t^{\#}, f^{\#}]$, on prendra toujours l'état avec le $t^{\#}$ plus petit dans les petites zones (de dimension identique $\delta_1 \times \delta_2$) de l'espace d'états.

1) En observant que la solution optimale est une partition des tâches (en deux sous-ensembles séquencés dans l'ordre de Jackson, c'est-à-dire, l'ordre décroissant des q_i) où chaque partition doit être ordonnancée sur une machine, proposer un algorithme de programmation dynamique *A* pour résoudre ce problème.
Indications : Essayer de vous inspirer des exemples vus en cours. On pourra prendre comme états générés par l'algorithme des vecteurs $[t, f]$ avec t la date de fin de la dernière tâche sur la première machine et f la date de livraison maximale de toutes tâches ordonnancées.

Jackson: ordre ↘ des q_i :



$1 // L_{max}$: 2 machines.

La décis° est: choisir la machine sur laquelle placer la tâche. Le but étant de minimiser le L_{max}, on peut dire que les états sont de la forme suivante:

$$[t, f]$$

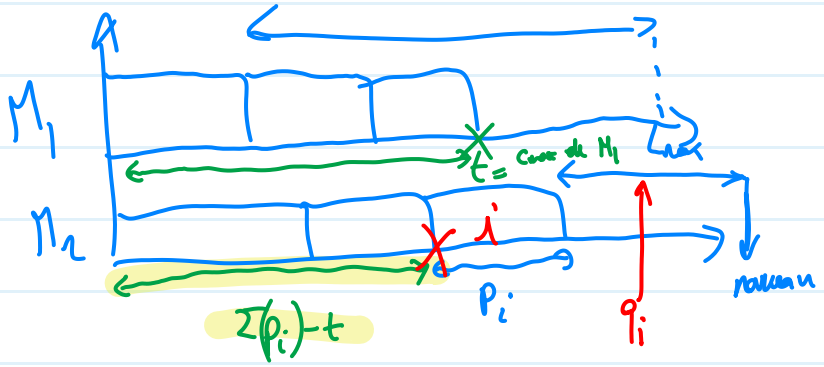
avec $\begin{cases} t : C_{max} \text{ sur } M_1 \\ f : L_{max} \text{ global cumulé} \end{cases}$

La décis° est donc:

$$[t, f] \xrightarrow{\text{sur } M_1} [t+p_i, \max\{f, t+p_i+q_i\}].$$

$$[t, f] \xrightarrow{\text{sur } M_2} [t, \max\{f, \underbrace{\sum_{k \in I}(p_k) - t + p_i + q_i}_{I = \text{tâches placées}}\}]$$

On peut supposer que les tâches sont triées sur l'ordre de Jackson.



2) Proposer des limites supérieures pour t et f pour toute solution réalisable.
Expliquer comment on peut éventuellement améliorer la complexité de la programmation dynamique.

Prog. Dyn : Algo. exacte.

$X_0 = \{[0,0]\}$

Pour $i = 1 .. n$ faire
  $X_i := \{\}$

  Pour chq état $[t,f] \in X_{i-1}$ faire
    Ajouter $[t+p_i, \max\{f, t+p_i+q_i\}]$ à X_i
    Ajouter $[t, \max\{f, \sum p_i > t + p_i + q_i\}]$ à X_i
  fin

  Supprimer X_{i-1}

  $t \in [1,P]$ dans P états, construits par X_i. | Pour chq valeur de t faire
    On garde un seul état avec f minimal
  fin
fin

$O(2^n)$ sans partir en range.

$O(n.P)$, $P = \sum p_i$
↳ on a enlevé les états dominés

Retourner $\min\{f\}$
  $[t,f] \in X_n$

$\Rightarrow \begin{cases} t \leq P \\ f \leq P + q_{max}. \end{cases}$

3) On considère l'heuristique H consistant à ordonnancer toutes les tâches sur la première machine. Montrer que $H \leq 3.OPT$ (avec OPT la valeur de la solution optimale).

$\Rightarrow$ On met tout sur M_1.

$\Rightarrow OPT \geq C_{max}(OPT) \geq \frac{P}{2}$ et $H \leq P + q_{max}$

On note que $OPT = \max\{L_j^*\} \geq C_i + q_i = C_i + q_{max} \geq q_{max} \Rightarrow OPT \geq L_{max}$.

$H \leq P + q_{max}$
$\leq 2OPT + OPT$
$H \leq 3OPT.$

4) En convertissant la programmation dynamique, montrer que le problème admet un FPTAS (Fully Polynomial Time Approximation Scheme).
Indications : Il faudrait considérer des états approchés $[t^{\#}, f^{\#}]$ générés par une version tronquée de l'algorithme de programmation dynamique, appelée $A^{\#}$. On pourra montrer qu'à chaque itération i de ces algorithmes, un état $[t, f]$ généré par A possède un état homologue $[t^{\#}, f^{\#}]$ généré par $A^{\#}$ avec les conditions suivantes :

$$t - i.\delta_1 \leq t^{\#} \leq t$$

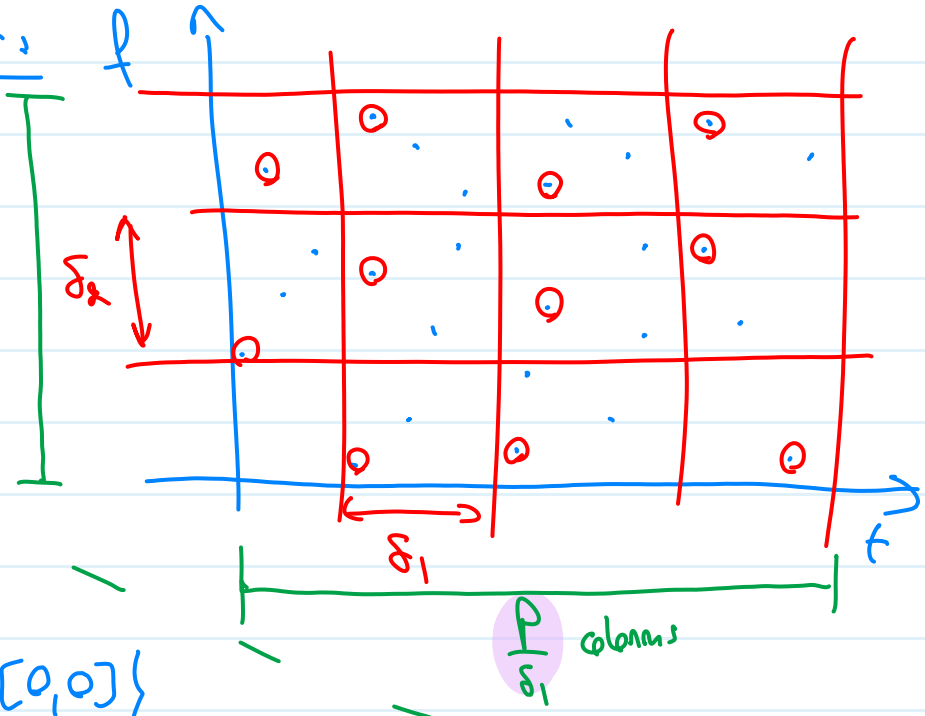
$$f^{\#} \leq f + i.\delta_2 + i.\delta_1$$

avec $\delta_1 = \varepsilon P/4N$ et $\delta_2 = \varepsilon H/6N$ (noter que P est la somme de toutes les durées opératoires et que ε est la précision de l'algorithme $A^{\#}$).

Dans la phase de remplacement des états $[t, f]$ par $[t^{\#}, f^{\#}]$, on prendra toujours l'état avec le $t^{\#}$ plus petit dans les petites zones (de dimension identique $\delta_1 \times \delta_2$) de l'espace d'états.



On obtient :



On prend le point qui minimise t.

FPTAS :

$\mathcal{X}_0^\# = \{[0,0]\}$

Pour $i = 1 \dots n$ faire
  $\mathcal{X}_i^\# = \{\}$

  Pour chq état $[t,f] \in \mathcal{X}_{i-1}^\#$ faire
    Ajouter $[t+p_i, \max\{f, t+p_i+q_i\}]$ à $\mathcal{X}_i^\#$
    Ajouter $[t, \max\{f, t+p_i+q_i\}]$ à $\mathcal{X}_i^\#$
  fin

  Supprimer $\mathcal{X}_{i-1}^\#$

  Pour chq rectangle faire
    Garder l'état avec $t^\#$ minimal
  fin
fin

Retourner $\min\{f\}$
$[t,f] \in \mathcal{X}_n^\#$

maintient $\frac{H}{\delta_2} \cdot \frac{P}{\delta_1}$ états au max = nb cellules du tableau

$O\left(n \cdot \frac{P}{\delta_1} \cdot \frac{H}{\delta_2}\right)$

$= O\left(n \cdot \frac{4n}{\varepsilon} \cdot \frac{6n}{\varepsilon}\right)$

$= O\left(\frac{n^3}{\varepsilon}\right)$

Lemme : $\forall [t,f] \in \mathcal{X}_i$, $\exists [t^\#, f^\#] \in \mathcal{X}_i^\#$ tq :

$$\begin{cases} t - i\delta_1 \leqslant t^\# \leqslant t \Rightarrow \\ f^\# \leqslant f + i\delta_1 + i\delta_2. \end{cases}$$

